

Python Scope

[< Previous](#)[Next >](#)

A variable is only available from inside the region it is created. This is called **scope**.

Local Scope

A variable created inside a function belongs to the *local scope* of that function, and can only be used inside that function.

Example

[Get your own Python Server](#)

A variable created inside a function is available inside that function:

```
def myfunc():  
    x = 300  
    print(x)  
  
myfunc()
```

[Try it Yourself »](#)

Function Inside Function

As explained in the example above, the variable `x` is not available outside the function, but it is available for any function inside the function:

Example

The local variable can be accessed from a function within the function:

```
def myfunc():  
    x = 300  
    def myinnerfunc():  
        print(x)  
    myinnerfunc()  
  
myfunc()
```

[Try it Yourself »](#)

ADVERTISEMENT

[REMOVE ADS](#)

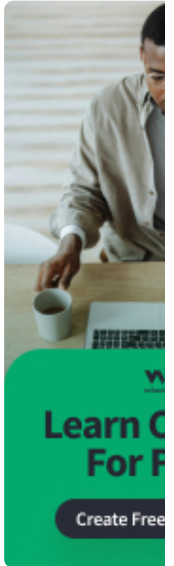
Global Scope

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

Example

A variable created outside of a function is global and



COLOR
PICKER



```
x = 300

def myfunc():
    print(x)

myfunc()

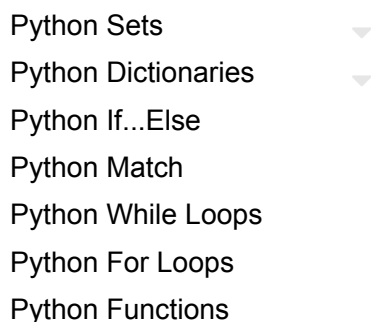
print(x)
```

Naming Variables

Example

```
x = 300

def myfunc():
    x = 200
```



Try it Yourself »

Python Decorators
Python Range
Python Lambda
Python Arrays
Python OOP
Python Classes/Objects
Python Inheritance
Python Iterators
Python Polymorphism
Python Scope
Python Modules
Python Dates
Python Math
Python JSON
Python RegEx
Python PIP
Python Try...Except
Python String Formatting
Python None
Python User Input
Python VirtualEnv

File Handling

Python File Handling
Python Read Files
Python Write/Create Files

Global Keyword

If you need to create a global variable, but are stuck in the local scope, you can use the `global` keyword.

The `global` keyword makes the variable global.

Example

If you use the `global` keyword, the variable belongs to the global scope:

```
def myfunc():  
    global x  
    x = 300  
  
myfunc()  
  
print(x)
```

[Try it Yourself »](#)

Also, use the `global` keyword if you want to make a change to a global variable inside a function.

Example

To change the value of a global variable inside a function, refer to the variable by using the `global` keyword:

```
x = 300  
  
def myfunc():  
    global x  
    x = 200  
  
myfunc()  
  
print(x)
```

Try it Yourself »

Nonlocal Keyword

The `nonlocal` keyword is used to work with variables inside nested functions.

The `nonlocal` keyword makes the variable belong to the outer function.

Example

If you use the `nonlocal` keyword, the variable will belong to the outer function:

```
def myfunc1():  
    x = "Jane"  
    def myfunc2():  
        nonlocal x  
        x = "hello"  
    myfunc2()  
    return x  
  
print(myfunc1())
```

Try it Yourself »

Exercise [?]

Consider the following code: